UNC

DR. HUMBERTO FERNÁNDEZ-MORÁN

UNIVERSIDAD NACIONAL DE LAS CIENCIAS

SCIENTIA VITAE

DR. HUMBERTO FERNÁNDEZ-MORÁN

CARRERA: **INTELIGENCIA ARTIFICIAL**

CATEDRA: **LOGICA Y ALGORITMO**

SECCIÓN : **Nº 21**

# OMNIVISION

VERSIÓN : **1.0**


integrantes del Equipo:

Albert Fonseca

Emanuel Jimenez

Tutor:

Ing. Deyanira Jimenez


04 / 2026

UNC
DR. HUMBERTO FERNÁNDEZ-MORÁN

# I N D I C E


1 Descripción, Propósito y justificación del proyecto socio tecnológico 3

2. Objetivos: General y Específicos 4

3. Descripción Técnica: Herramientas de Diseño a implementar en el desarrollo del Proyecto 5

4. Marco legal del proyecto socio tecnológico 8

5. Alcances del proyecto socio tecnológico 11

6. Limitaciones 13

7. Enfoque Metodológico del proyecto / PMBOK 14

8. Estructura de Desglose de Trabajo ( EDT ) 17

9. Diagrama de Gantt 20

10. Referencias bibliográficas 23

# OMNIVISION

**Sistema de Monitoreo** de **Videovigilancia Inteligente** en **Tiempo Real basado** en **Edge Computing** y **Visión Artificial** para el Análisis de **Comportamiento Humano y Generación de Alertas Tempranas.**

## 1. DESCRIPCIÓN, PROPÓSITO Y JUSTIFICACIÓN DEL PROYECTO SOCIO TECNOLÓGICO

El proyecto **Omni**Vision surge de la necesidad de fortalecer la seguridad en entornos compartidos mediante el uso de herramientas tecnológicas que actúen de forma preventiva. El **"porqué"** de esta propuesta reside en la vulnerabilidad de las comunidades y comercios ante incidentes que, por falta de supervisión constante, no se detectan a tiempo. Al integrar **IA y visión artificial,** el proyecto busca **reducir la incertidumbre y el riesgo**, proporcionando un sistema que no solo observa, sino que entiende el comportamiento humano para identificar situaciones de peligro potencial antes de que escalen.

El valor esperado de esta solución es la creación de un entorno más seguro y confiable para la comunidad. Al procesar los datos bajo el esquema de **Edge Computing**, el sistema ofrece una respuesta inmediata y local, lo que garantiza que las alertas lleguen en el momento justo para ser útiles. Este enfoque aporta un valor diferenciador al optimizar los recursos tecnológicos existentes, permitiendo que instituciones y locales comerciales cuenten con una herramienta de alta gama que no requiere de infraestructuras de red costosas o excesivamente complejas para operar con eficacia.

Desde la perspectiva de la unidad curricular el Proyecto Socio Tecnológico, **Omni**Vision se justifica como un ejercicio de soberanía tecnológica. El uso de lenguajes como **Python y JavaScript ( NestJS / FastAPI )** nos permiten desarrollar una solución personalizada que responde a las dinámicas específicas de nuestra localidad, evitando la dependencia de software privativo extranjero. El proyecto cumple con el objetivo de aplicar conocimientos técnicos avanzados para resolver un problema crítico del entorno social.

Fomentando la innovación dentro de la comunidad estudiantil y profesional. Finalmente, la arquitectura de microservicios propuesta asegura que el proyecto sea factible y escalable. La separación entre el procesamiento de imágenes ( FastAPI ) y la gestión de usuarios (NestJS) permite que el sistema crezca de acuerdo a las necesidades de la comunidad, pudiendo extenderse desde un pequeño comercio hasta un espacio público con múltiples puntos de análisis. Esta flexibilidad, sumada al uso de modelos de código abierto como YOLOv8, garantiza que la solución pueda mantenerse y evolucionar en el tiempo, cumpliendo con el propósito socio-tecnológico de generar un impacto positivo y duradero.

## 2. OBJETIVOS: GENERALES Y ESPECÍFICOS

### OBJETIVO GENERAL

Implementar un sistema de monitoreo de videovigilancia inteligente en tiempo real basado en Edge Computing y visión artificial, que permita identificar situaciones de riesgo mediante el análisis de comportamiento humano y emitir alertas tempranas automáticas para fortalecer la seguridad en entornos comerciales, institucionales y espacios públicos.

### OBJETIVOS ESPECÍFICOS

– Desarrollar un motor de análisis visual en tiempo real mediante el modelo YOLOv8 y algoritmos de rastreo (Yolo-movement) para la detección precisa de objetos y la identificación de patrones conductuales sospechosos.

– Configurar una arquitectura de Edge Computing utilizando FastAPI y procesamiento paralelo para ejecutar la inferencia de IA en nodos locales, garantizando una latencia mínima y una respuesta inmediata ante eventos detectados.

– Construir una plataforma de gestión centralizada con NestJS que orqueste la lógica de negocio, la administración de usuarios y la distribución eficiente de alertas hacia los dispositivos de control.

– Optimizar la transmisión de datos mediante streaming de video para permitir la supervisión remota de los incidentes en tiempo real, asegurando un uso eficiente del ancho de banda mediante la segmentación de eventos críticos.

– Validar la factibilidad y precisión del sistema mediante pruebas de campo en entornos reales (locales comerciales o espacios públicos), ajustando los parámetros de IA para minimizar falsos positivos y maximizar el valor socio–tecnológico de la solución.

## 3. DESCRIPCIÓN TECNICA.

### TECNOLOGÍAS A UTILIZAR (STACK TECNOLÓGICO)

El ecosistema de **Omni**Vision utiliza herramientas de alto rendimiento diseñadas para el manejo masivo de datos y ejecución de modelos de IA:

**YOLOv8 (You Only Look Once):** Es el pilar fundamental del sistema. Se utiliza esta versión por su equilibrio superior entre velocidad y precisión, permitiendo la detección y clasificación de múltiples objetos en milisegundos.

**Yolo–movement:** Librería especializada que trabaja sobre las detecciones de YOLO para realizar el seguimiento de trayectorias (tracking). Es lo que permite al sistema "entender" la dirección y persistencia de un sujeto en la escena.

**Python & FastAPI:** Utilizados para construir el motor de inferencia en el borde. FastAPI permite la ejecución de procesamiento paralelo, lo que es vital para analizar varios flujos de video de forma simultánea sin pérdida de cuadros (frames). NestJS: Actúa como el orquestador del backend, manejando la lógica de negocio, la seguridad de los datos y la distribución de alertas a través de microservicios robustos y escalables.

**OpenCV:** Biblioteca encargada del preprocesamiento de video, permitiendo la captura, el reescalado y el tratamiento de los flujos de imagen antes de ser analizados por la IA.

**Figma:** Herramienta principal para el diseño de interfaces de usuario del Dashboard de Monitoreo y el Panel de Configuración de OmniVision. Permite la creación de prototipos interactivos de alta fidelidad, sistemas de diseño reutilizables (componentes, tokens de color y tipografía).

**Adobe Photoshop:** Utilizado para el tratamiento y optimización de los assets gráficos del sistema, incluyendo la edición de capturas de pantalla del análisis de comportamiento, la creación de mockups de la interfaz en contexto real.

**Adobe Illustrator:** Empleado para la creación de los elementos vectoriales del sistema y creacion de logotipo, íconos del panel de alertas, diagramas de arquitectura técnica (flujo Edge Computing → Backend → Dashboard), la identidad visual del proyecto OmniVision y los gráficos exportables.

**SISTEMA OPERATIVO:**

**Windows 10:** Windows 10 corresponde al entorno de desarrollo de los integrantes del equipo. El despliegue en producción (Omni-Box del cliente) se ejecuta sobre Linux, garantizando compatibilidad nativa con las dependencias de Python, FastAPI y los modelos YOLOv8

**CONTROL DE VERSIONES:**

**GitLab:** Plataforma de gestión de repositorios Git utilizada para el control de versiones del proyecto, la colaboración entre los integrantes del equipo y el seguimiento de cambios en el código del backend.

**ENTORNOS DE DESARROLLO INTEGRADO (IDES):**

**Visual Studio Code, Cursor y Antigravity IDE:** IDEs utilizados a lo largo del desarrollo del proyecto. VS Code actúa como editor principal para el frontend y backend; Cursor, con su IA nativa, acelera la generación y refactorización de código complejo; y Antigravity IDE complementa el desarrollo del motor de inferencia en Python, facilitando la depuración y optimización del procesamiento de video con YOLOv8.

## ARQUITECTURA: EDGE COMPUTING HÍBRIDO

Para lograr el funcionamiento en tiempo real, el proyecto implementa una arquitectura distribuida:

**Nodos de Borde (Edge):** Servidores locales que ejecutan los modelos de YOLO directamente en el sitio. Esto elimina la necesidad de subir video bruto a internet, garantizando privacidad y una latencia casi nula.

**Nube/Centro de Control:** Una capa centralizada donde NestJS gestiona las alertas confirmadas, permitiendo que un usuario supervise múltiples puntos geográficos desde una única interfaz.

## MÉTODO DE CATEGORIZACIÓN Y RECONOCIMIENTO DEL COMPORTAMIENTO

El sistema no solo identifica "qué" hay en la imagen, sino "qué está haciendo". Para ello, aplica el siguiente flujo:

**Detección y Clasificación:** YOLOv8 identifica personas y objetos en el entorno, asignándoles una categoría y un puntaje de confianza.

**Extracción de Trayectorias:** Mediante Yolo-movement, el sistema asigna un identificador único a cada persona y mapea su movimiento a través del tiempo, calculando velocidades y patrones de desplazamiento.

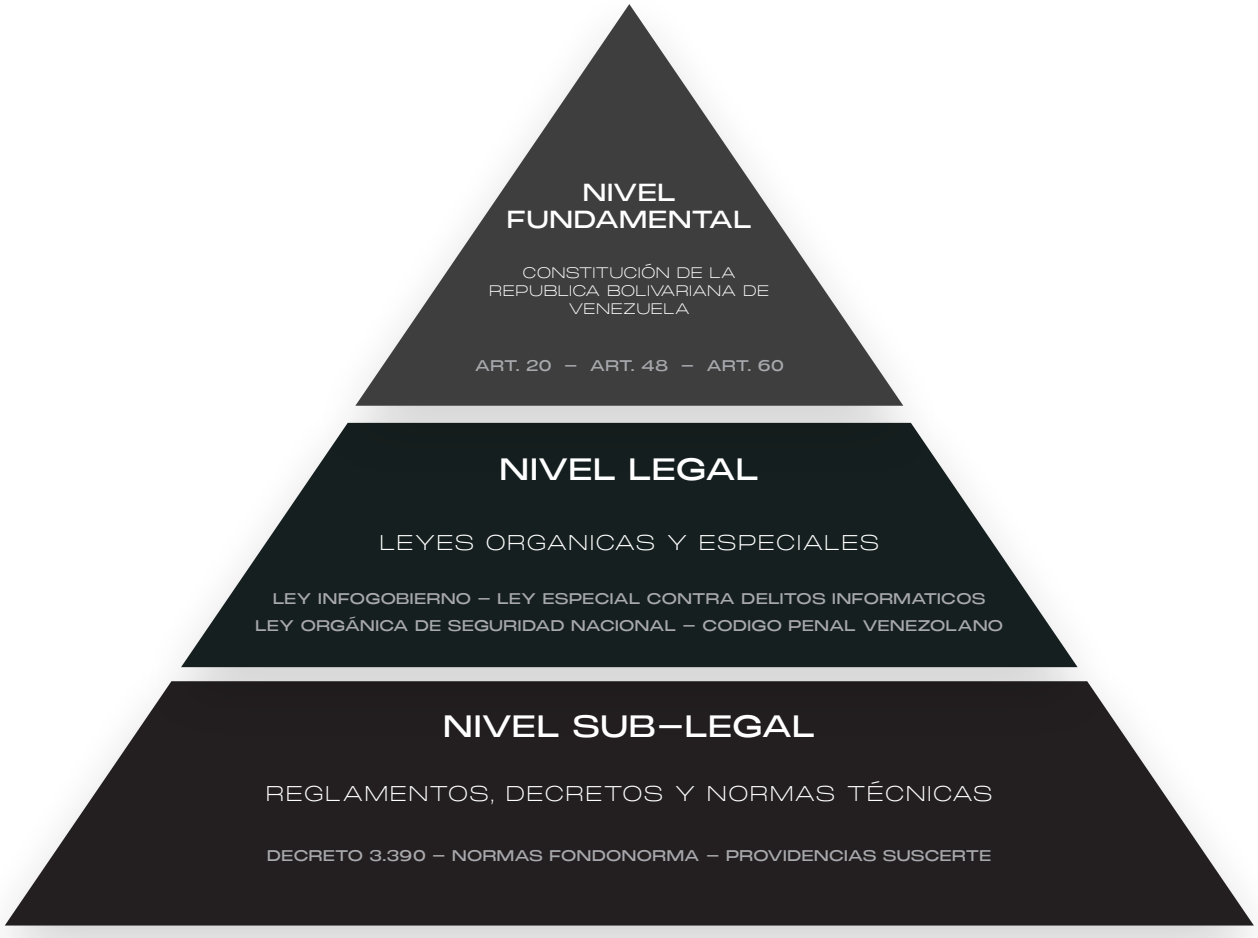
**Análisis de Comportamiento:** Se aplican reglas lógicas sobre el movimiento detectado. Por ejemplo:

> **Merodeo:** Un identificador que permanece en una zona crítica (como una caja fuerte o un pasillo restringido) más tiempo del umbral permitido.
>
> **Movimientos Erráticos:** Cambios bruscos de dirección o velocidad que no coinciden con el flujo normal de personas en el local.
>
> **Intrusión:** Presencia de un objeto de clase "persona" en horarios o áreas prohibidas.

GENERACIÓN DE ALERTAS

Cuando el análisis de comportamiento categoriza una **acción como sospechosa,** el **sistema activa un evento automático.** Se utiliza procesamiento paralelo para extraer el clip de video del incidente y enviarlo vía streaming al personal de seguridad, permitiendo una verificación humana inmediata y una respuesta efectiva ante el posible robo.

## 4. MARCO LEGAL ( PIRÁMIDE DE KELSEN )



### NIVEL FUNDAMENTAL – CONSTITUCIÓN DE LA REPÚBLICA BOLIVARIANA DE VENEZUELA (CRBV)

Es la base que garantiza los derechos ciudadanos que el proyecto debe respetar y proteger.

**Artículo 20:** Derecho al libre desenvolvimiento de la personalidad. El sistema debe operar sin discriminar ni coartar la libertad de los ciudadanos.

**Artículo 48:** Garantiza el secreto e inviolabilidad de las comunicaciones privadas. El flujo de datos y video debe estar cifrado (usando NestJS y protocolos seguros) para evitar filtraciones.

**Artículo 60:** Derecho a la protección de el honor, la vida privada, la intimidad, la propia imagen, confidencialidad y reputación. Este es el artículo más crítico: Omni-Vision justifica su uso en la seguridad, pero debe asegurar que las imágenes no se usen para fines distintos a la detección de delitos.

### NIVEL LEGAL – LEYES ORGÁNICAS Y ESPECIALES

Aquí se encuentran las normas que regulan el uso de la tecnología y la seguridad. Ley de Infogobierno: Promueve el uso de tecnologías libres. Al usar Python, FastAPI y NestJS, el proyecto se alinea con la soberanía tecnológica del Estado.

**Ley Especial contra los Delitos Informáticos:** Regula el acceso indebido a sistemas y la protección de datos. El proyecto debe cumplir con esta ley para asegurar que el almacenamiento de video en el Edge sea seguro y no vulnerable a hackeos.

**Ley Orgánica de Seguridad de la Nación:** Justifica la implementación de sistemas de monitoreo para la preservación de la paz pública y la protección de bienes institucionales y comerciales.

**Código Penal Venezolano:** Establece la base legal para la detección de robos y hurtos, que son los delitos que el sistema busca prevenir mediante las alertas tempranas.

## NIVEL SUB-LEGAL: REGLAMENTOS, DECRETOS Y NORMAS TÉCNICAS

Normativas específicas sobre el despliegue de tecnología.

**Decreto 3.390:** Dispone el uso prioritario de Software Libre en la Administración Pública Nacional. Al ser un proyecto socio-tecnológico basado en herramientas de código abierto (YOLOv8, OpenCV), cumple directamente con esta directriz.

**Normas FONDONORMA:** Relacionadas con los estándares de instalación de sistemas de seguridad y circuitos cerrados en locales comerciales y espacios públicos.

**Providencias de la SUSCERTE:** Sobre los estándares de criptografía y seguridad de la información que deben seguir los sistemas que manejan datos sensibles en el país.

## JUSTIFICACIÓN LEGAL DEL PROYECTO (RESUMEN PARA EL PST)

El proyecto Omni-Vision es legalmente factible bajo la pirámide de Kelsen porque:

**FINALIDAD:** Su propósito es la protección de los derechos ciudadanos y la propiedad (CRBV Art. 55 sobre el derecho a la protección por parte del Estado).

**SOBERANÍA:** Utiliza tecnologías abiertas, cumpliendo con los decretos presidenciales de desarrollo nacional.

**PRIVACIDAD:** El uso de Edge Computing permite que el video no salga del local o institución a menos que haya una alerta, lo que reduce la exposición de la imagen de las personas y respeta el Artículo 60 constitucional mejor que los sistemas basados en la nube tradicional.

**Nota Técnica:** Es vital incluir en el manual del proyecto que se deben colocar avisos visibles que informen que el área está siendo monitoreada por un sistema inteligente, cumpliendo así con el principio de transparencia informativa.

## 5. ALCANCES DEL PROYECTO

### ALCANCE DE INFRAESTRUCTURA Y HARDWARE (EDGE DEVICE)

**Provisión de Dispositivos de Procesamiento:** El proyecto incluye el diseño y ensamblaje de unidades de procesamiento local (basadas en micro–computadores como **Raspberry Pi** o hardware equivalente) encargadas de capturar los flujos de video de las cámaras existentes.

**Integración con Sistemas de Captura:** El dispositivo actuará como un nodo de borde capaz de conectarse a cámaras IP o analógicas (mediante adaptadores) para capturar frames en alta definición y procesarlos internamente con el modelo **YOLOv8.**

**Gestión de Cómputo Local:** Implementación de un entorno optimizado dentro del dispositivo para ejecutar **FastAPI** y el motor de IA, asegurando que el análisis de movimiento no dependa de la conexión a internet para funcionar.

### ALCANCE TÉCNICO (SOFTWARE Y RED)

**Despliegue de la "Omni–Box":** El sistema se entregará pre–instalado en el dispositivo, configurado para realizar el procesamiento paralelo de las señales de video y actuar como un filtro inteligente que solo envía datos relevantes a la nube.

**Sincronización con el Backend Central:** Configuración del canal de comu–nicación seguro entre el dispositivo físico y el servidor principal en NestJS, permitiendo el monitoreo remoto del estado del hardware y la recepción de alertas en tiempo real.

**Optimización de Recursos:** Ajuste del modelo de IA para que corra de forma eficiente en hardware con recursos limitados, garantizando la estabilidad del sistema 24/7.

## ALCANCE DEL SERVICIO Y VALOR AGREGADO

**Modelo de Infraestructura como Servicio (IaaS):** El cliente recibe el dispositivo físico y el acceso a la plataforma de monitoreo bajo una modalidad de suscripción o servicio gestionado, donde el desarrollador mantiene el control y actualización del hardware.

**Configuración de Escenarios de Seguridad:** Personalización de los algoritmos de Yolo-movement dentro del dispositivo para adaptarse a la disposición física del local o espacio público del cliente.

**Soporte y Monitoreo de Hardware:** El alcance incluye la supervisión del correcto funcionamiento de los dispositivos desplegados, asegurando que el servicio de alertas tempranas sea ininterrumpido.

## LIMITACIONES (FUERA DE ALCANCE)

**Propiedad Intelectual:** Tanto el diseño del software dentro del dispositivo como la lógica de los modelos de IA son propiedad exclusiva del desarrollador.

**Instalación de Cableado Estructural:** El alcance se limita a la conexión y configuración del dispositivo a la red existente; no incluye trabajos de obra civil o cableado extenso de cámaras.

**Autonomía Eléctrica:** El dispositivo requiere una fuente de alimentación estable; el respaldo energético **(UPS)** ante fallas eléctricas externas no forma parte del alcance inicial del proyecto.

## BENEFICIO ESTRATÉGICO PARA EL PROYECTO SOCIO TECNOLOGICO

**Viabilidad Económica:** Reduce los costos para el cliente al no necesitar servidores costosos, solo un microcomputador optimizado.

**Privacidad:** Los datos sensibles se quedan en el dispositivo físico del local, cumpliendo con la protección de imagen.

**Innovación:** Pasas de ser un desarrollador de software a un integrador de soluciones IoT (Internet of Things), lo cual tiene un peso académico muy alto en el área de ingeniería

## 6. LIMITACIONES

### LIMITACIONES TÉCNICAS Y DE HARDWARE

**Capacidad de Procesamiento en el Borde:** El rendimiento del análisis en tiempo real está sujeto a las especificaciones técnicas del dispositivo **(ej. Raspberry Pi).** La cantidad de cámaras que un solo dispositivo puede procesar simultáneamente estará limitada por su memoria **RAM** y capacidad de cómputo **(GPU/CPU).**

**Dependencia de Infraestructura Preexistente:** El sistema requiere que el cliente posea cámaras con protocolos de salida compatibles **(como RTSP)** y una red local estable para la captura del flujo de video. No se asume responsabilidad por fallas en el hardware previo del usuario.

**Condiciones Ambientales y de Iluminación:** La precisión del modelo **YOLOv8** puede verse afectada en entornos de extrema oscuridad, presencia de niebla densa o deslumbramientos directos hacia el lente, lo que podría aumentar el margen de error en la detección de comportamientos.

### LIMITACIONES OPERATIVAS Y DE SERVICIO

**Autonomía Eléctrica:** El funcionamiento continuo del dispositivo Omni-Box depende de un suministro eléctrico estable. El proyecto no incluye sistemas de respaldo energético (UPS o plantas eléctricas), por lo que cortes de luz interrumpirán el servicio de monitoreo.

**Alcance de la Alerta:** El sistema se limita a la detección y notificación temprana del evento sospechoso. La respuesta física (intervención de seguridad, detención de sujetos o contacto con entes policiales) queda fuera del alcance del software y es responsabilidad exclusiva del cliente.

**Mantenimiento de Conexión:** Aunque el procesamiento de IA es local **(Edge),** el envío de alertas al servidor NestJS y la visualización remota requieren de una conexión a internet activa. Si el servicio de internet del local falla, las alertas solo podrán almacenarse localmente pero no notificarse en tiempo real de forma remota.

### LIMITACIONES LEGALES Y ÉTICAS

**Privacidad y Biometría:** El sistema está diseñado para el análisis de comportamiento y no para el reconocimiento facial de identidad (biometría). Por lo tanto, no puede identificar nombres de personas ni cruzar datos con bases de datos civiles, respetando el marco de protección de imagen personal.

**Propiedad del Software:** El cliente tiene acceso al servicio y a la infraestructura, pero no a la manipulación del código fuente ni a la propiedad intelectual de los modelos de IA desarrollados, los cuales permanecen bajo la titularidad del autor del proyecto.

.

## 7. ENFOQUE METODOLÓGICO DEL PROYECTO / PMBOK

El proyecto adopta un enfoque híbrido. Utiliza la rigidez del PMBOK para la gestión de la infraestructura y el hardware, combinándola con metodologías ágiles (Scrum) para el desarrollo iterativo de los modelos de IA y el backend.

### INICIO: DEFINICIÓN DEL PROYECTO

En esta fase se formaliza la existencia del proyecto.

**Acta de Constitución:** Documento que define el propósito de Omni-Vision, los interesados (clientes comerciales, instituciones) y el director del proyecto.

**Identificación de Interesados:** Análisis de quiénes recibirán el servicio y cuáles son sus necesidades específicas de seguridad.

### PLANIFICACIÓN: EL PLAN DE GESTIÓN

Aquí se define el "cómo" se hará el trabajo.

**EDT (Estructura de Desglose del Trabajo):** Descomposición del proyecto en paquetes de trabajo (ej.: Configuración de Raspberry Pi, entrenamiento de YOLO, desarrollo de API en NestJS).

**Gestión del Cronograma:** Definición de los hitos críticos para asegurar que el sistema esté operativo en los tiempos acordados.

**Gestión de Riesgos:** Identificación de posibles problemas (ej.: falta de componentes electrónicos, latencia alta en el video o falsos positivos en la IA) y creación de planes de respuesta.

### EJECUCIÓN: DESARROLLO Y DESPLIEGUE

Es la fase donde se realiza el trabajo técnico.

**Adquisiciones:** Compra y ensamblaje de los dispositivos de borde (Raspberry Pi/Nodos Edge).

**Desarrollo Técnico:** Programación del motor de inferencia en FastAPI y la lógica de orquestación en NestJS.

**Integración de IA:** Implementación de YOLOv8 y Yolo-movement para el reconocimiento en tiempo real.

### MONITOREO Y CONTROL: CALIDAD DEL SERVICIO

Se asegura que el proyecto avance según lo planificado y que el sistema cumpla con los estándares.

**Control de Calidad:** Pruebas de detección de comportamiento para asegurar que las alertas se generen correctamente.

**Gestión del Alcance:** Validar que el dispositivo cumpla con las funciones prometidas (monitoreo, streaming y alerta) sin exceder los recursos del hardware.

**Monitoreo de Desempeño:** Supervisar la latencia del Edge Computing para que el análisis siga siendo en tiempo real.

### CIERRE: ENTREGA DEL SERVICIO

Fase final de la implementación del proyecto socio tecnológico.

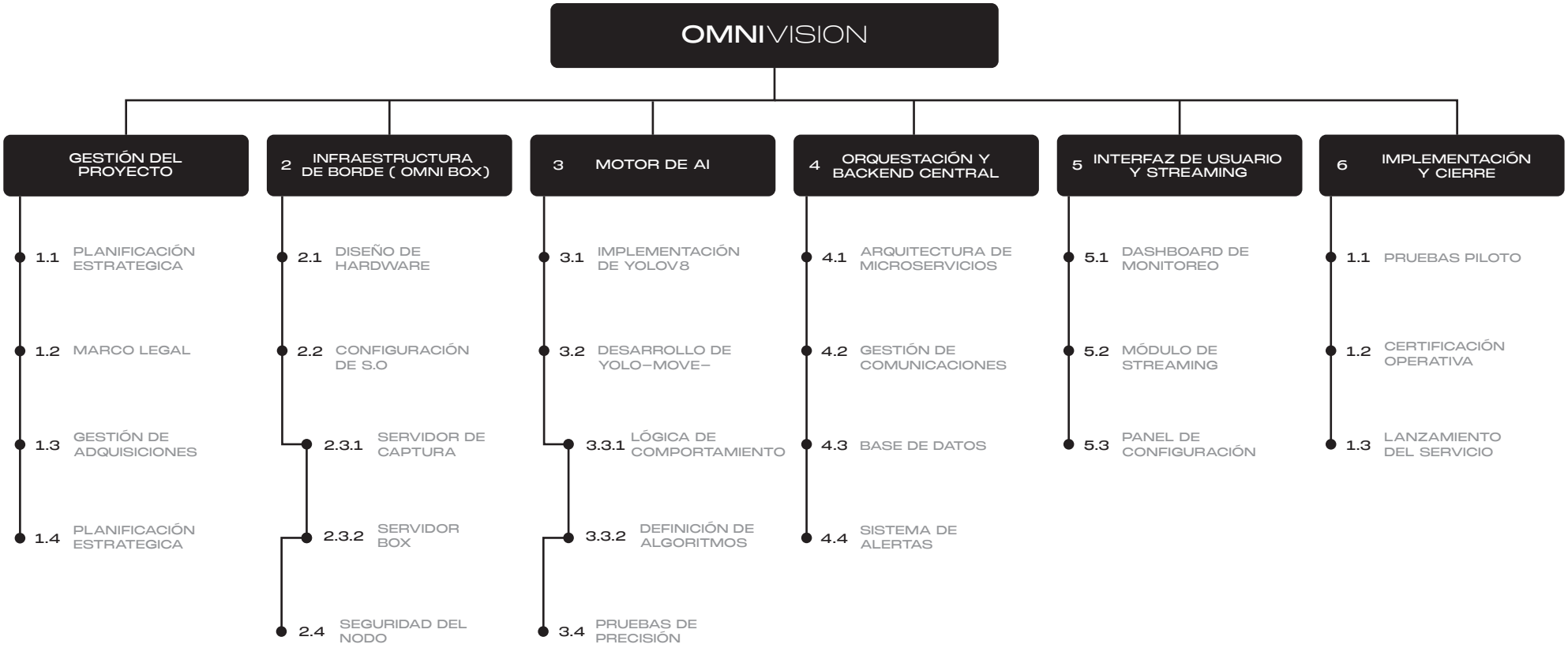
**Puesta en Marcha:** Instalación física de los dispositivos en el local o institución del cliente.

**Cierre de Adquisiciones:** Finalización de contratos con proveedores de hardware.

**Lecciones Aprendidas:** Documentación de los retos técnicos superados para futuros escalamientos del servicio.

.

## 8. ESTRUCTURA DE DESGLOSE DE TRABAJO (EDT)



## 1. GESTIÓN DEL PROYECTO

**1.1. Planificación Estratégica:** Definición de objetivos, cronograma y gestión de riesgos.

**1.2. Marco Legal:** Redacción de contratos de servicio y cumplimiento de la Pirámide de Kelsen.

**1.3. Gestión de Adquisiciones:** Compra y logística de hardware (Raspberry Pi, fuentes, carcasas).

## 2. INFRAESTRUCTURA DE BORDE (EDGE DEVICE – "OMNI-BOX")

**2.1. Diseño de Hardware:** Selección de componentes y diseño de refrigeración para el microcomputador.

**2.2. Configuración de S.O.:** Instalación de Linux optimizado y gestión de dependencias de IA.

**2.3. Nodo de Inferencia (FastAPI):**

**2.3.1. Implementación del servidor de captura de video (OpenCV).**

**2.3.2. Integración de procesamiento paralelo para flujos múltiples.**

**2.4. Seguridad del Nodo:** Encriptación de datos locales y protocolos de acceso seguro.

## 3. MOTOR DE INTELIGENCIA ARTIFICIAL

**3.1. Implementación de YOLOv8:** Configuración de pesos y optimización del modelo para hardware limitado.

**3.2. Desarrollo de Yolo-movement:** Programación de la lógica de tracking y persistencia de identificadores.

3.3. Lógica de Comportamiento:

3.3.1. Definición de algoritmos de detección de merodeo.

3.3.2. Configuración de perímetros de seguridad virtual (Zonas de interés).

**3.4. Pruebas de Precisión:** Ajuste de umbrales para reducción de falsos positivos.

## 4. ORQUESTACIÓN Y BACKEND CENTRAL (NESTJS)

**4.1. Arquitectura de Microservicios:** Configuración del núcleo central en NestJS.

**4.2. Gestión de Comunicaciones:** Implementación de WebSockets para alertas en tiempo real.

**4.3. Base de Datos:** Diseño de esquemas para logs de incidentes y perfiles de clientes.

**4.4. Sistema de Alertas:** Integración con servicios de notificación (Push, Telegram o Email).

## 5. INTERFAZ DE USUARIO Y STREAMING

**5.1. Dashboard de Monitoreo:** Desarrollo de la interfaz web para visualización de métricas.

**5.2. Módulo de Streaming:** Configuración de la transmisión segmentada de video de incidentes.

**5.3. Panel de Configuración:** Herramienta para que el administrador ajuste las reglas de comportamiento.

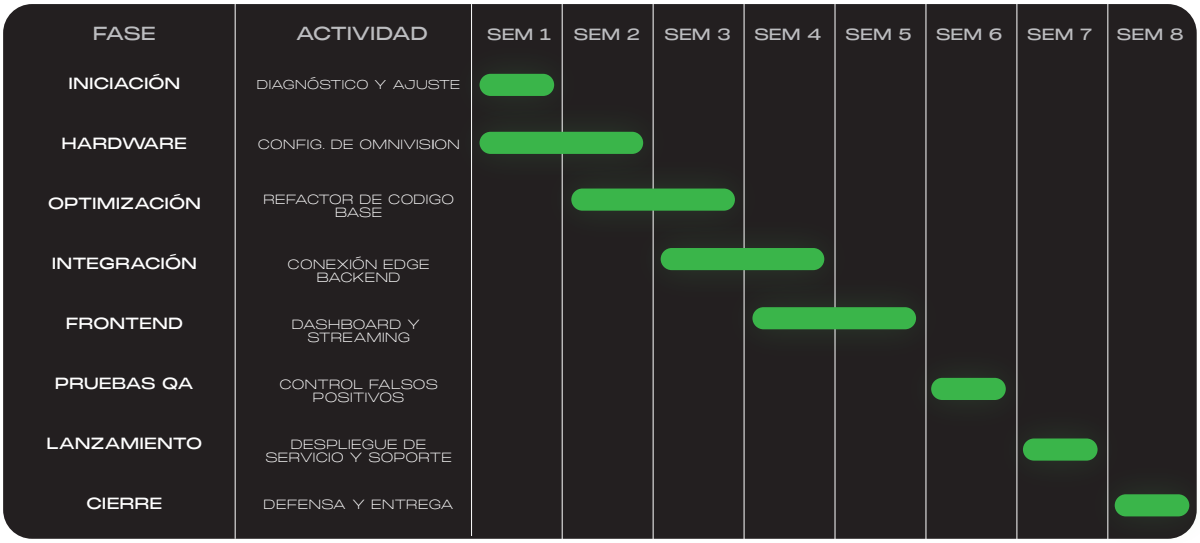
### 6. IMPLEMENTACIÓN Y CIERRE

**6.1. Pruebas Piloto:** Instalación y validación del servicio en el entorno real del cliente.

**6.2. Certificación Operativa:** Verificación de la estabilidad del sistema 24/7.

**6.3. Lanzamiento del Servicio:** Activación oficial de la infraestructura de monitoreo inteligente.

## 9. DIAGRAMA DE GANNT




### JUSTIFICACIÓN TÉCNICA DE LOS TIEMPOS DE EJECUCIÓN

El proyecto se ejecutará en un ciclo compacto de 8 semanas, fundamentado en tres pilares de eficiencia: el uso de un **MVP preexistente**, la asistencia de IA Generativa en el flujo de desarrollo y una arquitectura de Edge Computing modular.

### FASE I: CIMENTACIÓN (SEMANA 1)

**Descripción:** Ajuste del marco legal bajo la Pirámide de Kelsen y definición de requerimientos específicos del cliente.

**Justificación:** Al tener una visión clara del producto (Omni-Vision), el diagnóstico no requiere meses de investigación. Se utiliza la primera semana para blindar el proyecto legalmente, especialmente en el manejo de datos de video en Venezuela.

### FASE II: INFRAESTRUCTURA Y HARDWARE (SEMANAS 1-2)

**Descripción:** Preparación de la "Omni-Box" (Raspberry Pi/Edge Device) y configuración del sistema operativo.

**Justificación:** El uso de Docker permite una implementación inmediata. No se pierde tiempo configurando dependencias manualmente; se despliegan contenedores con el entorno de ejecución ya probado en el MVP, reduciendo el tiempo de configuración de hardware en un 60%.

### FASE III: OPTIMIZACIÓN DE IA Y TRAYECTORIAS (SEMANAS 2-3)

**Descripción:** Refactorización del código base de YOLOv8 e integración de la lógica de seguimiento (Yolo-movement).

**Justificación:** Esta es la fase donde la IA Generativa (Copilot/ChatGPT) actúa como multiplicador de fuerza. La generación de algoritmos para el cálculo de vectores de movimiento y detección de merodeo se acelera significativamente, permitiendo pasar de la detección simple al análisis de comportamiento en solo 15 días.

### FASE IV: INTEGRACIÓN DE SISTEMAS (SEMANAS 3–5)

**Descripción:** Sincronización del nodo Edge con el Backend en NestJS y ajustes en el Dashboard.

**Justificación:** Al existir ya un Front-end funcional, el trabajo se centra exclusivamente en la conexión de WebSockets para las alertas en tiempo real y la optimización del streaming. Se aprovecha el código base probado para evitar errores de arquitectura comunes.

### FASE V: VALIDACIÓN Y DESPLIEGUE (SEMANAS 6–8)

**Descripción:** Control de calidad (QA), pruebas de campo y entrega del servicio.

**Justificación:** Se asigna un tiempo crítico para la mitigación de falsos positivos. La semana 6 es vital para ajustar los umbrales de sensibilidad de la IA en el entorno real del cliente, asegurando que el servicio sea con-fiable antes de la defensa final en la semana 8.

## 10. REFERENCIAS BIBLIOGRÁFICAS

**García, R. y Martínez, L. (2021).** Implementación de microservicios en Node.js para la gestión de datos en tiempo real bajo la Ley de Infogobierno. Trabajo Especial de Grado no publicado, Universidad Nacional Experimental Politécnica de la Fuerza Armada, Caracas, Venezuela.

**Jocher, G., Chaurasia, A. y Qiu, J. (2023).** Ultralytics YOLO (Versión 8.0) [Software]. Recuperado de https://github.com/ultralytics/ultralytics

**Presidencia de la República Bolivariana de Venezuela. (2004, diciembre 28).** Decreto Nº 3.390: Uso prioritario de Software Libre en la Administración Pública Nacional. Gaceta Oficial Nº 38.095.

**Universidad Central de Venezuela. (2023).** Desarrollo de sistemas de visión artificial para la asistencia en entornos urbanos. Trabajo de Grado de Maestría no publicado, Facultad de Ciencias, Postgrado en Ciencias de la Computación, Caracas, Venezuela.

**Wang, Y. y Chen, J. (2022).** Edge computing for real-time video analysis: A survey. IEEE Access, 10, 15432–15450. https://doi.org/10.1109/ACCESS.2022.3145678